Лабораторна робота **№ 3**

З дисципліни **«Розробка Інтернет клієнт-серверних систем»**

**Виконав:**

Студент 3 курсу

3 групи ІПЗ

Сідун Владислав Владиславович

**2026**

**Тема:** Функції, рекурсія та замикання в Go

**Мета:** Ознайомитися з механізмами створення та використання функцій у мові програмування Go, а також дослідити особливості застосування рекурсії та замикань під час розробки програм.

# Виконання

**Варіант 11:** 3, 6, 10, 11, 17, 31, 36, 38, 40, 42, 47, 48

```
package main

import (
  "fmt"
  "math"
  "strings"
  "unicode"
)

func main() {
  fmt.Println("==================================================
")
  fmt.Println("   ЛАБОРАТОРНА РОБОТА №3 | ВАРІАНТ
11            ")
  fmt.Println("==================================================
")

  Task3()
  divider()

  Task6()
  divider()

  Task10()
  divider()

  Task11()
```

```
  divider()

  Task17()
  divider()

  Task31()
  divider()

  Task36()
  divider()

  Task38()
  divider()

  Task40()
  divider()

  Task42()
  divider()

  Task47()
  divider()

  Task48()

  fmt.Println("\n==================================================
==")
  fmt.Println("              ВИКОНАННЯ
ЗАВЕРШЕНО                 ")
  fmt.Println("==================================================
")
}

func divider() {
  fmt.Println("\n--------------------------------------------------
--")
}
```

```go
// ==============================================
// ЗАВДАННЯ 1: ФУНКЦІЇ
// ==============================================

// sumTwoSlices повертає суму всіх елементів двох зрізів цілих
чисел.
func sumTwoSlices(a []int, b []int) int {
  sum := 0
  for _, v := range a {
    sum += v
  }
  for _, v := range b {
    sum += v
  }
  return sum
}

// Завдання 3: Сума елементів двох зрізів
func Task3() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №3 (Функції - Сума двох
зрізів)")
  slice1 := []int{1, 2, 3, 4, 5}
  slice2 := []int{10, 20, 30}
  fmt.Printf("Вхідні дані:\n - Зріз 1: %v\n - Зріз 2: %v\n",
slice1, slice2)
  result := sumTwoSlices(slice1, slice2)
  fmt.Printf("Результат (сума елементів обох зрізів): %d\n",
result)
}

// findKeyByValue шукає в map[int]string ключ за значенням-рядком
str.
// Повертає вказівник на ключ та true якщо знайдено, або nil та
false якщо ні.
func findKeyByValue(m map[int]string, str string) (*int, bool) {
  for key, val := range m {
```

```go
    if val == str {
      k := key // зберігаємо в локальну змінну, щоб повернути
вказівник
      return &k, true
    }
  }
  return nil, false
}

// Завдання 6: Пошук ключа в map за значенням-рядком
func Task6() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №6 (Функції - Пошук ключа
в Map)")
  myMap := map[int]string{1: "apple", 2: "banana", 3: "cherry"}
  searchStr := "banana"
  fmt.Printf("Вхідні дані:\n - Map: %v\n - Рядок для пошуку:
'%s'\n", myMap, searchStr)
  key, found := findKeyByValue(myMap, searchStr)
  if found {
    fmt.Printf("Результат: Знайдено за ключем %d\n", *key)
  } else {
    fmt.Println("Результат: nil (значення не знайдено)")
  }
}

// cubeNumber підносить передане число в куб і повертає
результат.
func cubeNumber(n float64) float64 {
  return n * n * n
}

// Завдання 10: Піднесення числа в куб
func Task10() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №10 (Функції - Куб
числа)")
  num := 4.0
  fmt.Printf("Вхідні дані: число = %.1f\n", num)
```

```go
  result := cubeNumber(num)
  fmt.Printf("Результат (число в кубі): %.2f\n", result)
}

// isAnagram перевіряє, чи є два рядки анаграмами одне одного.
// Анаграма — рядки з однаковими літерами в будь-якому порядку.
func isAnagram(s1, s2 string) bool {
  s1 = strings.ToLower(s1)
  s2 = strings.ToLower(s2)
  if len(s1) != len(s2) {
    return false
  }
  charCount := make(map[rune]int)
  for _, ch := range s1 {
    charCount[ch]++
  }
  for _, ch := range s2 {
    charCount[ch]--
    if charCount[ch] < 0 {
      return false
    }
  }
  return true
}

// Завдання 11: Перевірка двох рядків на анаграму
func Task11() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №11 (Функції - Анаграма)")
  str1 := "listen"
  str2 := "silent"
  fmt.Printf("Вхідні дані:\n - Рядок 1: '%s'\n - Рядок 2:
'%s'\n", str1, str2)
  result := isAnagram(str1, str2)
  fmt.Printf("Результат (є анаграмою?): %v\n", result)
}

// countUpperCase підраховує кількість великих літер у рядку.
```

```go
// Використовує unicode.IsUpper для коректної роботи з UTF-8 символами.
func countUpperCase(s string) int {
  count := 0
  for _, ch := range s {
    if unicode.IsUpper(ch) {
      count++
    }
  }
  return count
}

// Завдання 17: Підрахунок великих літер у рядку
func Task17() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №17 (Функції - Великі літери)")
  input := "Hello World from Go!"
  fmt.Printf("Вхідні дані: '%s'\n", input)
  result := countUpperCase(input)
  fmt.Printf("Результат (кількість великих літер): %d\n", result)
}

// ==============================================
// ЗАВДАННЯ 2: РЕКУРСІЯ
// ==============================================

// fibRecursive рекурсивно обчислює n-те число Фібоначчі.
// Базові випадки: fib(0) = 0, fib(1) = 1.
// Рекурсивний крок: fib(n) = fib(n-1) + fib(n-2).
func fibRecursive(n int) int {
  if n == 0 {
    return 0
  }
  if n == 1 {
    return 1
  }
  return fibRecursive(n-1) + fibRecursive(n-2)
```

```go
}

// Завдання 31: Число Фібоначчі (рекурсія)
func Task31() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №31 (Рекурсія - Число
Фібоначчі)")
  n := 10
  fmt.Printf("Вхідні дані: n = %d\n", n)
  result := fibRecursive(n)
  fmt.Printf("Результат (fib(%d)): %d\n", n, result)
}

// extractBetweenBrackets рекурсивно витягує підрядок між першою
парою дужок.
// collecting=true означає що ми вже всередині дужок і збираємо
символи.
func extractBetweenBrackets(s string, collecting bool) string {
  if len(s) == 0 {
    return ""
  }
  ch := s[0]
  rest := s[1:]
  if ch == '(' {
    // Знайшли відкриваючу дужку — починаємо збір символів
    return extractBetweenBrackets(rest, true)
  }
  if ch == ')' {
    // Знайшли закриваючу дужку — зупиняємо збір
    return ""
  }
  if collecting {
    // Ми всередині дужок — додаємо поточний символ до результату
    return string(ch) + extractBetweenBrackets(rest, true)
  }
  // Ще не зустріли відкриваючу дужку — пропускаємо символ
  return extractBetweenBrackets(rest, false)
}
```

```go
// Завдання 36: Витягти рядок між дужками (рекурсія)
func Task36() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №36 (Рекурсія - Вміст між
дужками)")
  input := "hello(world)end"
  fmt.Printf("Вхідні дані: '%s'\n", input)
  result := extractBetweenBrackets(input, false)
  fmt.Printf("Результат (між дужками): '%s'\n", result)
}

// sumNegativeRecursive рекурсивно обчислює суму від'ємних
елементів зрізу.
// Базовий випадок: зріз порожній — повертаємо 0.
func sumNegativeRecursive(slice []int) int {
  if len(slice) == 0 {
    return 0
  }
  first := slice[0]
  rest := slice[1:]
  if first < 0 {
    return first + sumNegativeRecursive(rest)
  }
  return sumNegativeRecursive(rest)
}

// Завдання 38: Сума від'ємних елементів зрізу (рекурсія)
func Task38() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №38 (Рекурсія - Сума
від'ємних)")
  slice := []int{3, -5, 7, -2, -8, 4, 1}
  fmt.Printf("Вхідні дані: %v\n", slice)
  result := sumNegativeRecursive(slice)
  fmt.Printf("Результат (сума від'ємних елементів): %d\n",
result)
}
```

```go
// sumDivisibleByN рекурсивно обчислює суму елементів зрізу,
кратних n.
// Базовий випадок: зріз порожній — повертаємо 0.
func sumDivisibleByN(slice []int, n int) int {
  if len(slice) == 0 {
    return 0
  }
  first := slice[0]
  rest := slice[1:]
  if n != 0 && first%n == 0 {
    return first + sumDivisibleByN(rest, n)
  }
  return sumDivisibleByN(rest, n)
}

// Завдання 40: Сума елементів зрізу кратних n (рекурсія)
func Task40() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №40 (Рекурсія - Сума
кратних n)")
  slice := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
  n := 3
  fmt.Printf("Вхідні дані:\n - Зріз: %v\n - n = %d\n", slice, n)
  result := sumDivisibleByN(slice, n)
  fmt.Printf("Результат (сума елементів кратних %d): %d\n", n,
result)
}

// ==============================================
// ЗАВДАННЯ 3: ЗАМИКАННЯ
// ==============================================

// makeSquareChain приймає початкове число n і повертає функцію-
замикання.
// Кожен виклик замикання підносить поточне значення n у квадрат
(n = n²).
// Змінна n зберігається між викликами завдяки механізму
замикання.
```

```go
func makeSquareChain(n float64) func() float64 {
  return func() float64 {
    n = math.Pow(n, 2) // n захоплено замиканням — стан
зберігається між викликами
    return n
  }
}

// Завдання 42: Послідовне піднесення в квадрат через замикання
func Task42() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №42 (Замикання -
Послідовний квадрат)")
  startValue := 2.0
  fmt.Printf("Вхідні дані: початкове число = %.1f\n", startValue)
  square := makeSquareChain(startValue)
  fmt.Println("Результат:")
  fmt.Printf(" - Виклик 1 (2²):   %.0f\n", square())
  fmt.Printf(" - Виклик 2 (4²):   %.0f\n", square())
  fmt.Printf(" - Виклик 3 (16²):  %.0f\n", square())
}

// makeCharCounter приймає рядок і повертає функцію-замикання.
// Замикання "запам'ятовує" рядок і при кожному виклику
// рахує кількість входжень переданого символу в цей рядок.
func makeCharCounter(s string) func(rune) int {
  // s захоплено замиканням — зберігається в пам'яті між
викликами
  return func(ch rune) int {
    count := 0
    for _, c := range s {
      if c == ch {
        count++
      }
    }
    return count
  }
}
```

```go
// Завдання 47: Лічильник входжень символу через замикання
func Task47() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №47 (Замикання - Лічильник
символів)")
  text := "hello world"
  fmt.Printf("Вхідні дані: рядок = '%s'\n", text)
  counter := makeCharCounter(text)
  fmt.Println("Результат:")
  fmt.Printf(" - Символ 'l': %d разів\n", counter('l'))
  fmt.Printf(" - Символ 'o': %d разів\n", counter('o'))
  fmt.Printf(" - Символ 'z': %d разів\n", counter('z'))
}

// makeBitChecker приймає номер біта і повертає функцію-
замикання.
// Замикання перевіряє, чи встановлений заданий біт у переданому
числі в одиницю.
// Перевірка побітовою операцією: (num >> bitPos) & 1 == 1
func makeBitChecker(bitPos int) func(int) bool {
  // bitPos захоплено замиканням
  return func(num int) bool {
    return (num>>bitPos)&1 == 1
  }
}

// Завдання 48: Перевірка встановленого біта через замикання
func Task48() {
  fmt.Println("---> ВИКОНАННЯ ЗАВДАННЯ №48 (Замикання - Перевірка
біта)")
  bitPosition := 3 // перевіряємо 3-й біт (нумерація з нуля)
  fmt.Printf("Вхідні дані: номер біта = %d\n", bitPosition)
  checkBit := makeBitChecker(bitPosition)
  // 12 у бінарному вигляді: 1100 — 3-й біт встановлений (1)
  // 5  у бінарному вигляді: 0101 — 3-й біт не встановлений (0)
  num1, num2 := 12, 5
  fmt.Println("Результат:")
```

```go
  fmt.Printf(" - Число %2d (бінарно: %04b), біт №%d = 1? ->
%v\n", num1, num1, bitPosition, checkBit(num1))
  fmt.Printf(" - Число %2d (бінарно: %04b), біт №%d = 1? ->
%v\n", num2, num2, bitPosition, checkBit(num2))
}
```

```
==================================================
   ЛАБОРАТОРНА РОБОТА №3 | ВАРІАНТ 11
==================================================
---> ВИКОНАННЯ ЗАВДАННЯ №3 (Функції - Сума двох зрізів)
Вхідні дані:
 - Зріз 1: [1 2 3 4 5]
 - Зріз 2: [10 20 30]
Результат (сума елементів обох зрізів): 75

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №6 (Функції - Пошук ключа в Map)
Вхідні дані:
 - Map: map[1:apple 2:banana 3:cherry]
 - Рядок для пошуку: 'banana'
Результат: Знайдено за ключем 2

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №10 (Функції - Куб числа)
Вхідні дані: число = 4.0
Результат (число в кубі): 64.00

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №11 (Функції - Анаграма)
Вхідні дані:
 - Рядок 1: 'listen'
 - Рядок 2: 'silent'
Результат (є анаграмою?): true

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №17 (Функції - Великі літери)
Вхідні дані: 'Hello World from Go!'
Результат (кількість великих літер): 3

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №31 (Рекурсія - Число Фібоначчі)
Вхідні дані: n = 10
Результат (fib(10)): 55

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №36 (Рекурсія - Вміст між дужками)
Вхідні дані: 'hello(world)end'
Результат (між дужками): 'world'

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №38 (Рекурсія - Сума від'ємних)
Вхідні дані: [3 -5 7 -2 -8 4 1]
Результат (сума від'ємних елементів): -15
```

```
--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №40 (Рекурсія - Сума кратних n)
Вхідні дані:
 - Зріз: [1 2 3 4 5 6 7 8 9 10]
 - n = 3
Результат (сума елементів кратних 3): 18

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №42 (Замикання - Послідовний квадр
Вхідні дані: початкове число = 2.0
Результат:
 - Виклик 1 (2²):   4
 - Виклик 2 (4²):   16
 - Виклик 3 (16²):  256

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №47 (Замикання - Лічильник символі
Вхідні дані: рядок = 'hello world'
Результат:
 - Символ 'l': 3 разів
 - Символ 'o': 2 разів
 - Символ 'z': 0 разів

--------------------------------------------------
---> ВИКОНАННЯ ЗАВДАННЯ №48 (Замикання - Перевірка біта)
Вхідні дані: номер біта = 3
Результат:
 - Число 12 (бінарно: 1100), біт №3 = 1? -> true
 - Число  5 (бінарно: 0101), біт №3 = 1? -> false

==================================================
              ВИКОНАННЯ ЗАВЕРШЕНО
==================================================
```

# Посилання на репозиторій

https://github.com/VladSSidun/Client_Server_Arch

# Відповіді на контрольні запитання

**1. Поясніть синтаксис оголошення функції в Go. Які елементи може містити сигнатура функції та чим вона відрізняється від сигнатур функцій у мовах C/C++ або Java?**

Функція в Go оголошується за допомогою ключового слова func, після якого вказується ім'я функції, список параметрів у дужках та тип значення, що повертається. Наприклад: func add(a int, b int) int. Головна відмінність від C/C++ та Java полягає в тому, що тип параметра вказується після його імені, а не перед ним. Крім того, Go підтримує повернення одразу кількох значень з однієї функції, що є унікальною особливістю мови. Також підтримуються іменовані результати та variadic-параметри (довільна кількість аргументів через ...).

**2. У чому полягає різниця між передаванням аргументів у функцію за значенням і передаванням посилань (через вказівники) у Go?**

При передаванні за значенням функція отримує копію змінної — будь-які зміни всередині функції не впливають на оригінал. При передаванні через вказівник (pointer) функція отримує адресу змінної в пам'яті і може змінювати оригінальне значення. Передавання за значенням доцільно використовувати для простих типів (int, float, bool), коли зміна оригіналу небажана. Вказівники використовують тоді, коли потрібно змінити оригінальну змінну, або коли передається великий об'єкт (наприклад, struct) і копіювання є витратним.

**3. Що таке функція з множинними значеннями, що повертаються (multiple return values) у Go? Які переваги має такий підхід під час обробки помилок?**

Go дозволяє функції повертати одразу декілька значень, що вказуються через кому в круглих дужках після параметрів: func divide(a, b float64) (float64, error). Це особливо зручно для обробки помилок: прийнятий у Go підхід полягає в тому, що функція повертає результат і помилку одночасно. Це усуває потребу у винятках (exceptions) як у Java, робить обробку помилок явною та передбачуваною, а код — легшим для читання і відлагодження.

**4. Поясніть принцип роботи іменованих результатів функції у Go. У яких випадках їх використання є доцільним?**

Іменовані результати — це можливість заздалегідь дати ім'я значенням, що повертаються: func split(sum int) (x, y int). Такі змінні автоматично оголошуються на початку функції і можуть використовуватися в її тілі. Виклик return без аргументів поверне їх поточні значення (naked return). Іменовані результати доцільно використовувати у коротких функціях для підвищення читабельності коду, а також для документування призначення кожного з повернутих значень.

**5. Що таке рекурсія? Опишіть структуру рекурсивної функції та поясніть роль базового випадку у запобіганні нескінченній рекурсії.**

Рекурсія — це техніка програмування, при якій функція викликає саму себе для розв'язання підзадачі меншого розміру. Структура рекурсивної функції обов'язково містить два елементи: базовий випадок (base case) — умова, за якої рекурсія зупиняється і повертається конкретне значення; та рекурсивний виклик — виклик функції з аргументом, що наближає задачу до базового випадку. Без базового випадку функція викликала б себе нескінченно, що призвело б до переповнення стеку викликів і аварійного завершення програми (stack overflow).

**6. Порівняйте рекурсивні та ітеративні алгоритми. У яких випадках рекурсія є більш зручною або ефективною?**

Ітеративні алгоритми використовують цикли і, як правило, є більш ефективними за пам'яттю, оскільки не створюють додаткових кадрів стеку при кожному виклику. Рекурсія є більш природною та зручною для задач із вираженою ієрархічною структурою: обхід дерев, обчислення чисел Фібоначчі, факторіалу, пошук у вкладених структурах даних. Рекурсивний код часто коротший і легший для розуміння, однак при великій глибині викликів може призвести до переповнення стеку.

**7. Що таке замикання у Go? Поясніть, як замикання дозволяють функції зберігати доступ до змінних із зовнішньої області видимості.**

Замикання (closure) — це анонімна функція, яка "захоплює" змінні із зовнішньої області видимості і зберігає доступ до них навіть після того, як зовнішня функція завершила своє виконання. У Go замикання створюються автоматично, коли анонімна функція посилається на змінну із навколишнього контексту. Ця змінна продовжує існувати в пам'яті доти, доки існує саме замикання, що дозволяє зберігати стан між викликами.

**8. Поясніть різницю між звичайною функцією, анонімною функцією та замиканням у Go.**

Звичайна функція оголошується з іменем на рівні пакета і доступна у всьому пакеті. Анонімна функція — це функція без імені, яка оголошується безпосередньо у виразі і може бути одразу викликана або збережена у змінній. Замикання — це анонімна функція, яка додатково захоплює і використовує змінні з навколишнього контексту. Тобто кожне замикання є анонімною функцією, але не кожна анонімна функція є замиканням — лише та, що посилається на зовнішні змінні.

**9. Як працює лексична область видимості у Go та яку роль вона відіграє у створенні замикань?**

Лексична область видимості означає, що функція має доступ до змінних того блоку коду, в якому вона була оголошена, а не того, де вона викликається. Саме це є основою механізму замикань: анонімна функція оголошена всередині іншої функції "бачить" її локальні змінні, і це зв'язування відбувається на етапі компіляції. Завдяки лексичній області видимості замикання точно знає, які саме змінні воно захоплює, що робить поведінку програми передбачуваною.

**10. Які потенційні проблеми можуть виникати при використанні рекурсії та замикань у Go?**

При використанні рекурсії головна проблема — переповнення стеку викликів (stack overflow) при надто великій глибині рекурсії. Go не виконує оптимізацію хвостової рекурсії, тому для великих значень n краще використовувати ітеративний підхід. При використанні замикань у циклах може виникнути поширена помилка: всі замикання захоплюють одну й ту саму змінну-лічильник циклу за посиланням, а не за значенням. Щоб уникнути цього, потрібно передавати змінну як параметр у замикання. Крім того, замикання, що утримує великий об'єкт, може запобігти його звільненню збирачем сміття, що призведе до витоку пам'яті.

**Висновки**

У ході виконання лабораторної роботи було реалізовано 12 завдань варіанту 11, які охоплюють три основні теми: функції, рекурсія та замикання в мові програмування Go.

У розділі функцій було реалізовано операції над зрізами та map, перевірку анаграм, підрахунок великих літер і піднесення в куб. Було відпрацьовано синтаксис оголошення функцій, передавання параметрів та повернення кількох значень, зокрема вказівника та булевого значення одночасно.

У розділі рекурсії реалізовано обчислення числа Фібоначчі, витягування підрядка між дужками, суму від'ємних елементів та суму елементів кратних n. Кожна рекурсивна функція містить явний базовий випадок та рекурсивний крок.

У розділі замикань реалізовано послідовне піднесення в квадрат зі збереженням стану, лічильник символів у рядку та перевірку встановленого біта. Було наочно продемонстровано, як анонімна функція захоплює змінну із зовнішнього контексту і зберігає її між викликами.